



# Module 11: Basic Algorithm Analysis

CPE010 DATA STRUCTURES AND ALGORITHMS

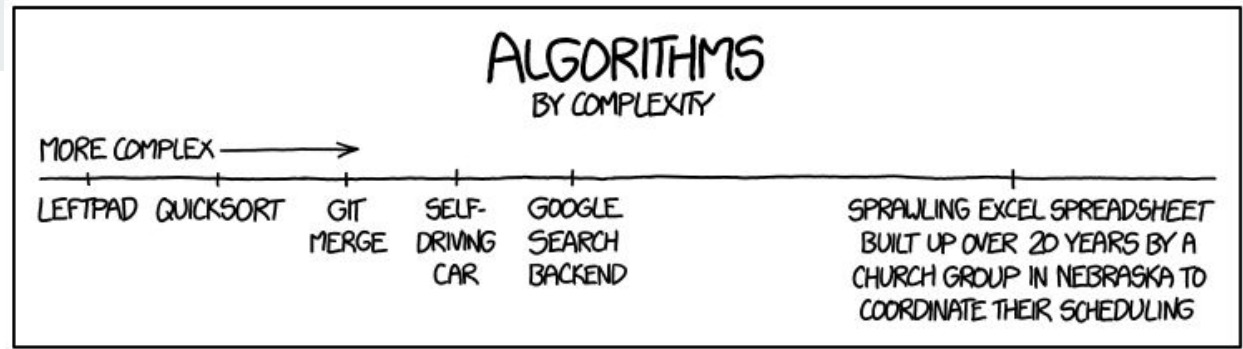


# Intended Learning Outcomes

After this module, the student should be able to:

- Measure the runtime of algorithms using theoretical analysis
- Analyze the results of runtime performance by comparing experimental and theoretical analysis

# Introduction



- An algorithm has important dimensions we need to measure:
  - Space Complexity
  - Time Complexity
- Comparison allows us to select from alternative approaches to a solution.
  - Allows the consideration of constraints before coding.
- Analysis is performed **before** coding; Profiling (also known as benchmarking) is performed **after** coding.



# Measuring Runtime

- Absolute runtime **cannot be predicted** but we can measure an algorithm's **relative runtime**.
  - Analyze the algorithm as a function of the input items **N**.
  - This N could be *number of items in the container, length of a string, degree of polynomials or digits of an integer*.
- Counting the **frequency of operations** or the number of **abstract operations** as a function n.
  - Abstract operations are also called **steps**.
- We will take a look at some examples.



We can measure runtime in two ways:

- Experimental (aka empirical) analysis
- Theoretical Analysis

**EXPERIMENTAL ANALYSIS IS TOO DEPENDENT ON HARDWARE.**

**However, THEORETICAL ANALYSIS is MACHINE INDEPENDENT.**

Theoretical analysis is performed through the counting methods. Experimental analysis is benchmarking/profiling.



## Example 1: Printing each element in an array

```
for (var i = 0; i < a.length; i++)  
{  
    std::cout << arr[i];  
}
```

- The value of N in this example is **the array length**.
- Other operations in the code are shown in the table below:

|                                  |                 |
|----------------------------------|-----------------|
| Initialization of i              | Happens 1 time  |
| Comparison of i against a.length | Happens N times |
| Incrementing i                   | Happens N times |
| Array indexing                   | Happens N times |
| Printing                         | Happens N times |

The frequency of  
operations is  
 **$T(n) = 4n+1$**



## Example 2: Printing every other element

```
for (var i = 0; i < a.length; i += 2) {  
    std::cout << a[i];  
}
```

If N is still a.length, what is the frequency of operations now?



## Example 2: Printing every other element

```
for (var i = 0; i < a.length; i += 2) {  
    std::cout << a[i];  
}
```

If N is still a.length, what is the frequency of operations now?

*Give your answers before the answer is given.*



## Example 2: Printing every other element

```
for (var i = 0; i < a.length; i += 2) {  
    std::cout << a[i];  
}
```

If  $N$  is still  $a.length$ , what is the frequency of operations now?

**$T(n) = n/2$**  because we are printing a value every other  $N$ , which is  $N/2$ .



## Example 3: Getting the last element of an array

```
if (a.length > 0) {  
    return a[a.length - 1];  
} else {  
    error;  
}
```

Regardless of how big the array is,  
this will always take one step... therefore, **T(N) is what?**



## Example 3: Getting the last element of an array

```
if (a.length > 0) {  
    return a[a.length - 1];  
} else {  
    error;  
}
```

**$T(N)$  is 1.**

**Next, let's check loops!**

## Example 4: Printing a multiplication table

```
for (var i = 0; i < n; i++) {  
    for (var j = 0; j < n; j++) {  
        std::cout << i*j;  
    }  
    std::cout << std::endl;  
}
```

Outer loop executes **n times**.

Inside the loop, **n+1 times**.

- **n times for loop**
- **1 for std::endl**

What is the count of abstract operations / frequency of operations?  
**Provide answer in chat.**

## Example 4: Printing a multiplication table

```
for (var i = 0; i < n; i++) {  
    for (var j = 0; j < n; j++) {  
        std::cout << i*j;  
    }  
    std::cout << std::endl;  
}
```

Outer loop executes **n times**.

Inside the loop, **n+1 times**.

- **n times for loop**
- **1 for std::endl**

$T(n) = n(n+1)$ , better written as  $T(n) = n^2 + n$

**Preferred expression!**

## C++ Variable Sizes



### Space Complexity

- Basically, defined as the amount of memory space the algorithm or problem instance consumes during runtime.
  - Must consider space for input values.
- How do we perform space complexity analysis? Check table.

| Type               | Size (in bytes) | Range                         |
|--------------------|-----------------|-------------------------------|
| char               | 1               | -127 to 127 or 0 to 255       |
| unsigned char      | 1               | 0 to 255                      |
| int                | 4               | -2147483648 to 2147483647     |
| unsigned int       | 4               | 0 to 4294967295               |
| short int          | 2               | -32768 to 32767               |
| unsigned short int | 2               | 0 to 65,535                   |
| long int           | 4               | -2147483648 to 2147483647     |
| unsigned long int  | 4               | 0 to 4294967295               |
| float              | 4               | +/- 3.4e +/- 38 (~7 digits)   |
| double             | 8               | +/- 1.7e +/- 308 (~15 digits) |

NOTE: Due to modern computers having large memory, space complexity is not always considered. However, for real-time systems, space complexity is still an important parameter; memory space is a major constraint.



## Example 1: Addition of numbers

```
{  
  int a = x + y + z;  
  return (a);  
}
```

There are 4 INTEGER-type variables.  
An integer variable takes 4 bytes each.  
The return value will also take another 4 bytes.

Therefore,

$$4 * 4 + 4 = 20 \text{ bytes.}$$

This is what we call a **FIXED COMPLEXITY** because there is set number and types of variables.



## Example 2: Sum of all elements in an array

```
function sum_of_numbers(arr[],N) {  
    sum=0  
    for(i = 0 to N){  
        sum=sum+arr[i]  
    }  
    print(sum)  
}
```

What would be the **space complexity** in bytes?



## Example 2: Sum of all elements in an array

```
function sum_of_numbers(arr[],N) {  
    sum=0  
    for(i = 0 to N){  
        sum=sum+arr[i]  
    }  
    print(sum)  
}
```

The array size N will take **4 bytes** each:  $N * 4$  bytes.  
Sum is an integer, will take another 4 bytes.  
Variable i (the iterator) will take another 4 bytes.

The print function and for loop will take **auxiliary space**, we will assume a value of 4 bytes,

Therefore,

**$(N * 4) + 12$  bytes.**



## Example 3: Recursive Factorial of a Number

```
factorial(N) {  
    if(N<=1) {  
        return 1;  
    }else{  
        return (N*factorial(N-1));  
    }  
}
```

What is the **space complexity** in bytes?



## Example 3: Recursive Factorial of a Number

```
factorial(N) {  
    if(N<=1) {  
        return 1;  
    }else{  
        return (N*factorial(N-1));  
    }  
}
```

N will take 4 bytes.

The if-else and recursive call can be assumed as 4 bytes.

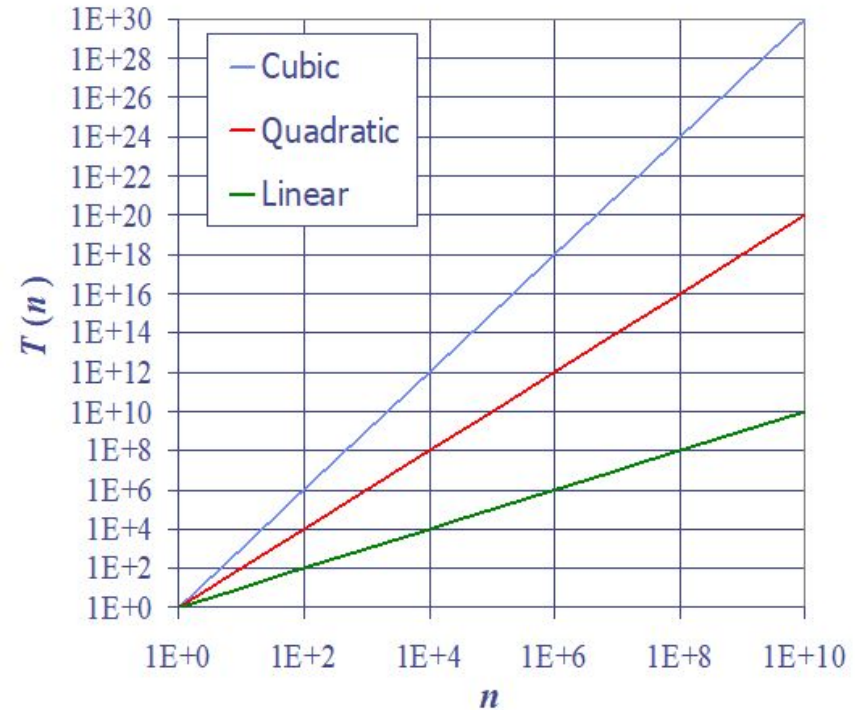
However, it is a recursive call so, we are going through the recursion up to the number of terms N. Therefore, the if-else and recursive call will be  $4*N$  bytes.

Space Complexity for the recursive factorial function will be  **$(4*N)+4$  bytes.**

# Time Complexity Classes

- Seven functions that often appear in algorithm analysis:
  - Constant  $\approx 1$
  - Logarithmic  $\approx \log n$
  - Linear  $\approx n$
  - N-Log-N  $\approx n \log n$
  - Quadratic  $\approx n^2$
  - Cubic  $\approx n^3$
  - Exponential  $\approx 2^n$

In a log-log chart, the slope of the line corresponds to the growth rate of the function

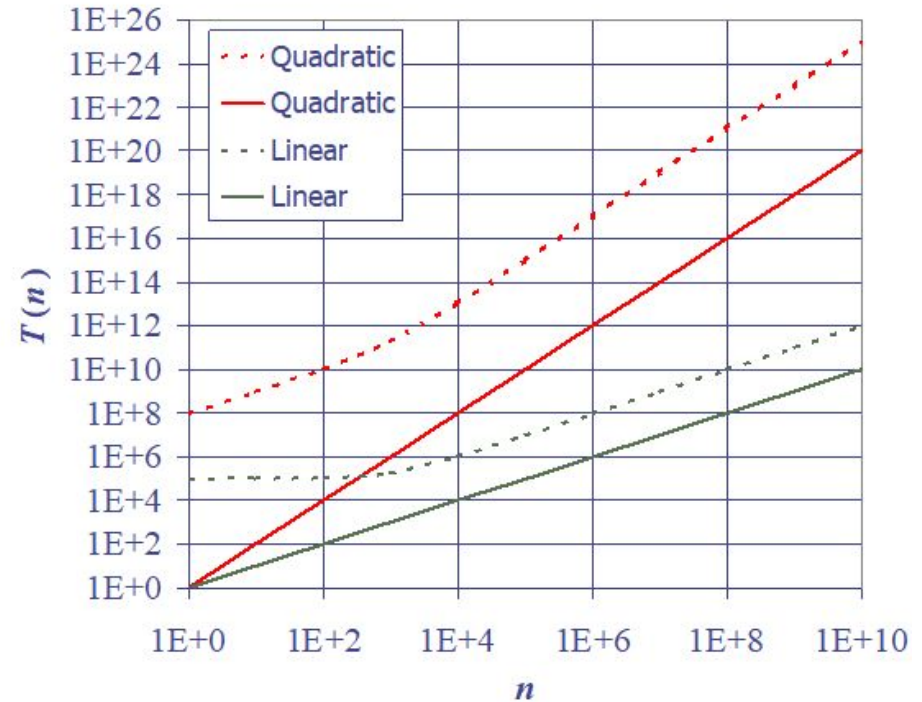


# Constant Factors

- The growth rate is **not affected** by:
  - Constant Factors or
  - Lower-order Terms

## Examples:

- $10^2n + 10^3$  is a linear function
- $10^3n^2 + 10^3n$  is a quadratic function





## Exploring Effects on Performance

In this section, we will explore the effects of different factors on performance considering the following:

- What if **input size increases**? How will this affect performance?
- What if you're **using a faster computer**? Will it be better?



## What happens when we increase input size?

| For an algorithm of complexity | If the input size doubles, then the running time |
|--------------------------------|--------------------------------------------------|
| 1                              | stays the same                                   |
| $\log n$                       | increases by a constant                          |
| $n$                            | doubles                                          |
| $n^2$                          | quadruples                                       |
| $n^3$                          | increases eight fold                             |
| $2^n$                          | is left as an exercise for the reader            |

# What are the effects of a faster computer?

Faster computers can solve larger problem sets in a fixed amount of time except when it is **exponential time algorithms**; then the improvement is negligible. See figure.

| For an algorithm of complexity | If you can solve a problem of this size on your 100MHz PC | Then on a 500MHz PC you can solve a problem set of this size | And on a supercomputer one thousand times faster than your PC you can solve a problem set of this size |
|--------------------------------|-----------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| $n$                            | 100                                                       | 500                                                          | 100000                                                                                                 |
| $n^2$                          | 100                                                       | 223                                                          | 3162                                                                                                   |
| $n^3$                          | 100                                                       | 170                                                          | 1000                                                                                                   |
| $2^n$                          | 100                                                       | 102                                                          | 109                                                                                                    |



## ONE TRILLION operations per second?

| $T(n)$ | 20                   | 40                   | 50                   | 60                             | 70                                 | 80                                 |
|--------|----------------------|----------------------|----------------------|--------------------------------|------------------------------------|------------------------------------|
| $n^5$  | 3.2<br>$\mu\text{s}$ | 102<br>$\mu\text{s}$ | 313<br>$\mu\text{s}$ | 778<br>$\mu\text{s}$           | 1.68<br>msec                       | 3.28<br>msec                       |
| $2^n$  | 1.05<br>msec         | 1.1<br>seconds       | 18.8<br>minutes      | 13.3<br>days                   | 37.4<br>years                      | 383<br>centuries                   |
| $3^n$  | 3.5<br>msec          | 4.7<br>months        | 227<br>centuries     | $1.3 \times 10^7$<br>centuries | $7.93 \times 10^{11}$<br>centuries | $4.68 \times 10^{16}$<br>centuries |

# ONE TRILLION operations per second?

| $T(n)$ | 20                   | 40                   | 50                   | 60                             | 70                                 | 80                                 |
|--------|----------------------|----------------------|----------------------|--------------------------------|------------------------------------|------------------------------------|
| $n^5$  | 3.2<br>$\mu\text{s}$ | 102<br>$\mu\text{s}$ | 313<br>$\mu\text{s}$ | 778<br>$\mu\text{s}$           | 1.68<br>msec                       | 3.28<br>msec                       |
| $2^n$  | 1.05<br>msec         | 1.1<br>seconds       | 18.8<br>minutes      | 13.3<br>days                   | 37.4<br>years                      | 383<br>centuries                   |
| $3^n$  | 3.5<br>msec          | 4.7<br>months        | 227<br>centuries     | $1.3 \times 10^7$<br>centuries | $7.93 \times 10^{11}$<br>centuries | $4.68 \times 10^{16}$<br>centuries |

## Takeaways:

- **HUGELY SIGNIFICANT** gap between polynomial and exponential time.
- **High-order polynomial problem instances can be solved in decent time, but difficult in problems with exponential time.**
- **REMEMBER:**
  - Polynomial time is **good, easy and tractable**.
  - Exponential time is **bad, hard and intractable**.



# Summary and Next Topics

- Theoretical Analysis
  - Time Complexity
  - Space Complexity
- Complexity Cases
- Effects of Different Factors on Algo Performance

Further details on asymptotic notations and amortized time complexity in the following modules.

---

# Thank you for participating!

Post your additional questions/queries in the module Q&A discussion post.